

ONEDRAFT

CAD-AI — Aria Powered

Application Runtime, Service Architecture & Execution Stack

Version 1.0

Status: Implementation Specification

Predecessor: Database Schema & OpenAPI Contract v0.1

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Execution Model: API + Domain Services + Workers + Event-Driven Jobs

Runtime Principle: Deterministic application services surrounding non-deterministic AI capabilities

1. APPLICATION STACK PRINCIPLE

OneDraft shall now be implemented as an executable distributed application rather than as a collection of independent utilities.

The application stack shall preserve the separation between:

API transport

application orchestration

domain logic

project model

persistence

geometry computation

AI orchestration

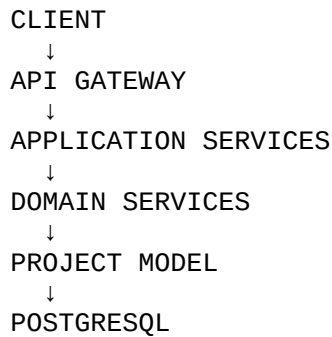
regulatory verification

documentation generation

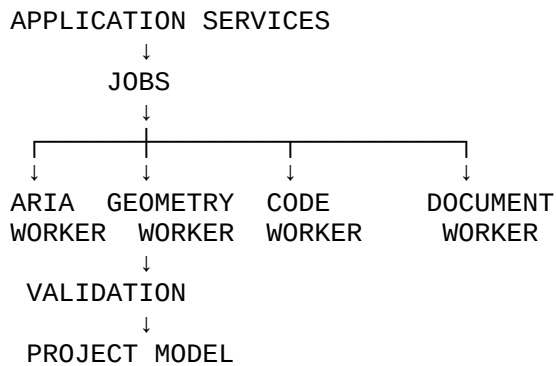
background execution

audit and provenance

The architectural flow becomes:



with computational workers operating alongside the transactional system:



2. APPLICATION LAYERS

The initial OneDraft application shall contain the following logical layers:

```
apps/  
  web/  
  api/  
  worker/  
  
packages/  
  domain/  
  application/  
  infrastructure/  
  geometry/  
  compliance/  
  documentation/  
  aria/  
  events/  
  contracts/
```

The architecture shall enforce directional dependencies.

The intended dependency direction is:

```
contracts  
↓
```

domain
↓
application
↓
infrastructure

Specialized computational packages depend upon domain contracts rather than bypassing the domain model.

3. WEB APPLICATION

The web application shall provide the primary user interface.

Initial responsibilities:

authentication
organization selection
project selection
project creation
project dashboard
requirements entry
model inspection
drawing inspection
redline interaction
validation results
document package review
acceptance

The web application shall not directly communicate with PostgreSQL.

All project mutations shall pass through the API.

4. API APPLICATION

The API application is the public application boundary.

Initial responsibilities:

authentication
authorization
request validation
idempotency
project routing
application command dispatch
query execution
job creation
job status
error normalization
audit context

The API shall not contain computational geometry algorithms.

The API shall not contain direct AI-provider logic.

The API shall not become the project model itself.

The API coordinates application operations.

5. WORKER APPLICATION

The worker application executes asynchronous computational operations.

Initial worker categories:

ARIA
GEOMETRY
COMPLIANCE
VALIDATION
DOCUMENTATION
EXPORT

Workers receive jobs through the application job system.

Workers must operate against explicit project and model versions.

6. DOMAIN PACKAGE

The domain package defines the canonical OneDraft semantic model.

Initial domain modules:

```
domain/  
  project.py  
  requirements.py  
  building.py  
  level.py  
  space.py  
  element.py  
  assembly.py  
  constraint.py  
  relationship.py  
  revision.py  
  command.py  
  drawing.py  
  redline.py  
  compliance.py  
  validation.py  
  document.py  
  acceptance.py
```

The domain package shall not depend upon:

FastAPI
PostgreSQL drivers
Redis
Celery
AI providers
browser frameworks
cloud providers

The domain model must remain executable independently of infrastructure.

7. APPLICATION PACKAGE

The application package contains use-case orchestration.

Initial services:

ProjectService
RequirementService
ModelService
CommandService
RevisionService
RedlineService
DrawingService
ComplianceService
ValidationService
DocumentService
AcceptanceService
JobService

Each service operates through explicit application commands and queries.

8. COMMAND / QUERY SEPARATION

OneDraft shall distinguish between:

Commands

and:

Queries

Commands mutate the project state.

Queries retrieve project state.

Examples:

CreateProject
CreateRequirement
CreateBuilding
CreateLevel

CreateSpace
CreateElement
ExecuteAriaCommand
CreateRevision
CreateRedline
ResolveRedline
RunValidation
GenerateDrawings
GenerateDocumentPackage
RecordAcceptance

Queries include:

GetProject
GetProjectModel
GetRevision
GetModelVersion
GetDrawingSet
GetValidationResults
GetRedlines
GetDocumentPackage
GetProjectHistory

9. COMMAND PIPELINE

Every project mutation shall pass through a standard execution pipeline.

```
REQUEST
↓
AUTHENTICATE
↓
AUTHORIZE
↓
VALIDATE INPUT
↓
LOAD PROJECT STATE
↓
VERIFY EXPECTED VERSION
↓
EVALUATE CONSTRAINTS
↓
EXECUTE DOMAIN OPERATION
↓
PERSIST CHANGE
↓
CREATE REVISION
↓
EMIT EVENT
↓
COMMIT
↓
RETURN RESULT
```

No application command shall bypass this boundary.

10. PROJECT CONTEXT

Every application operation affecting project state shall establish a ProjectContext.

Conceptually:

```
ProjectContext(  
    project_id,  
    organization_id,  
    actor_id,  
    revision_id,  
    model_version_id,  
    request_id  
)
```

The ProjectContext becomes the execution identity of the operation.

11. REQUEST CONTEXT

Every API request shall receive a RequestContext containing:

```
request_id  
actor_id  
organization_id  
project_id  
authorization_scopes  
timestamp  
idempotency_key
```

The RequestContext shall be propagated into application services and audit events.

12. DOMAIN COMMAND

The canonical internal command structure shall contain:

```
command_id  
command_type  
project_id  
actor_id  
expected_revision  
expected_model_version  
targets  
parameters  
constraints  
metadata
```

The command becomes the immutable description of an intended mutation.

13. COMMAND VALIDATION

Before execution, commands shall be validated at three levels.

Structural validation

Checks:

- required fields
- types
- UUID validity
- enumerations
- parameter formats

Domain validation

Checks:

- object existence
- object compatibility
- relationship validity
- model rules
- project state

Constraint validation

Checks:

- active constraints
- geometric constraints
- regulatory constraints
- coordination constraints
- document dependencies

A command that fails validation shall not mutate the project.

14. OPTIMISTIC CONCURRENCY

OneDraft shall use optimistic concurrency for model mutations.

Each command may specify:

- expected_revision
- expected_model_version

The service compares those values against current state.

If they differ:

REVISION_CONFLICT

shall be returned.

This prevents an operation created against an obsolete model state from silently overwriting newer work.

15. TRANSACTION SERVICE

The ProjectModelService established in the previous implementation stage becomes the primary transactional boundary.

Conceptually:

```
with project_transaction(context):
    state = repository.load(context.project_id)

    validate_command(state, command)

    result = domain.execute(state, command)

    repository.persist(result)

    revision = revision_service.create(result)

    event_service.record(result)

    transaction.commit()
```

The implementation shall guarantee atomicity of the relational model update.

16. REPOSITORY INTERFACES

The domain and application layers shall use repository interfaces.

Initial interfaces:

ProjectRepository
RequirementRepository
ModelRepository
ElementRepository
RevisionRepository
CommandRepository
DrawingRepository
RedlineRepository
ComplianceRepository
ValidationRepository
DocumentRepository
AcceptanceRepository
EventRepository

JobRepository

The PostgreSQL implementation resides in the infrastructure layer.

17. INFRASTRUCTURE PACKAGE

The infrastructure package contains external-system implementations.

Initial structure:

```
infrastructure/  
  database/  
  repositories/  
  transactions/  
  object_storage/  
  queue/  
  authentication/  
  telemetry/
```

No infrastructure implementation should become part of the domain model.

18. DATABASE ADAPTER

The PostgreSQL adapter shall provide:

```
connection management  
transaction management  
migration execution  
query execution  
repository persistence  
locking  
connection pooling
```

Connection credentials must be supplied through environment configuration or deployment secrets.

They shall never be embedded in source code.

19. OBJECT STORAGE

Large artifacts shall not be stored directly inside PostgreSQL.

Object storage shall contain:

```
geometry payloads  
generated PDFs  
drawing exports  
source regulatory documents
```

rendered previews
document packages
large analysis artifacts

PostgreSQL stores the authoritative references and hashes.

Conceptually:

PostgreSQL
↓
object_reference
↓
Object Storage

20. OBJECT HASHING

Stored artifacts shall receive cryptographic content hashes.

The hash becomes part of provenance.

For example:

geometry_hash
document_hash
source_hash
package_hash

The system can therefore determine whether an artifact has changed since it was recorded.

21. JOB SYSTEM

Asynchronous operations shall use a persistent job abstraction.

Initial job structure:

job_id
project_id
job_type
revision_id
model_version_id
status
priority
progress
attempt_count
payload
result
error
created_at
started_at
completed_at

Jobs shall be persisted independently of worker process memory.

22. JOB LIFECYCLE

The standard lifecycle is:

```
QUEUED
↓
RUNNING
↓
VALIDATING
↓
COMPLETED
```

Failure:

```
RUNNING
↓
FAILED
```

Cancellation:

```
QUEUED
↓
CANCELLED
```

Retries must not produce duplicate project mutations.

23. JOB IDEMPOTENCY

Every computational job shall contain a deterministic execution identity.

Conceptually:

```
project_id
operation
model_version_id
revision_id
input_hash
```

The combination may be used to detect duplicate work.

A repeated request for the same deterministic operation should return the existing job or result where appropriate.

24. ARIA ORCHESTRATION LAYER

Aria shall not directly mutate PostgreSQL.

Aria operates through the OneDraft command interface.

The architecture is:

```
USER REQUEST
↓
ARIA ORCHESTRATOR
↓
PROJECT CONTEXT
↓
MODEL QUERY
↓
REASONING
↓
COMMAND PROPOSAL
↓
COMMAND VALIDATOR
↓
DOMAIN SERVICE
↓
MODEL TRANSACTION
```

This is the primary AI safety and determinism boundary.

25. ARIA PROVIDER ADAPTER

The underlying AI model provider shall be abstracted.

Conceptually:

```
class AIProvider:
    def generate(self, request) -> AIResponse:
        ...
```

Aria orchestration shall depend upon this interface rather than a specific provider SDK.

This permits model providers to change without changing the OneDraft domain API.

26. ARIA CONTEXT ASSEMBLY

Before Aria proposes a modification, the orchestration layer shall construct a controlled context.

The context may include:

```
project requirements
current model state
```

relevant elements
relationships
constraints
current revision
applicable code manifest
existing drawings
redlines
validation results
user instruction

Only relevant project information should be supplied to the reasoning operation.

27. ARIA OUTPUT CONTRACT

Aria shall return structured output.

Conceptually:

```
{
  "intent": "DESIGN_MODIFICATION",
  "commands": [
    {
      "operation": "MOVE_ELEMENT",
      "target_objects": ["uuid"],
      "parameters": {
        "distance": 300,
        "unit": "mm",
        "direction": "NORTH"
      }
    }
  ],
  "reasoning_summary": "...",
  "constraints_to_preserve": ["uuid"],
  "validation_requirements": ["SPATIAL", "COORDINATION"]
}
```

Free-form AI text shall not constitute a model mutation.

28. ARIA CONFIDENCE

AI confidence may be recorded but shall not substitute for deterministic validation.

The architecture therefore distinguishes:

AI confidence

from:

domain validity

and:

regulatory compliance

A high-confidence AI proposal can still fail validation.

29. GEOMETRY SERVICE

The geometry subsystem shall provide computational geometry independently of Aria.

Initial responsibilities:

- point operations
- line operations
- polygon operations
- intersection
- offset
- projection
- bounding boxes
- containment
- distance
- area
- volume
- transformations
- boolean operations

The geometry service operates against explicit model versions.

30. GEOMETRY ENGINE BOUNDARY

The geometry engine shall receive:

- model_version_id
- element identifiers
- geometry inputs
- operation
- parameters

and return:

- geometry result
- geometry hash
- bounding information
- validation information

The geometry engine shall not determine project ownership, billing, authorization, or acceptance.

31. MODEL GRAPH

The OneDraft project model shall be treated as a dependency graph.

Conceptually:

PROJECT
↓
BUILDING
↓
LEVEL
↓
SPACE
↓
ELEMENT
↓
ASSEMBLY

Relationships provide cross-graph dependencies:

ELEMENT
↔ ELEMENT
ELEMENT
↔ SPACE
ELEMENT
↔ ASSEMBLY
ELEMENT
↔ CONSTRAINT

Changes propagate through this graph.

32. DEPENDENCY PROPAGATION

When an element changes, the application shall determine affected downstream objects.

For example:

WALL CHANGE
↓
ROOM BOUNDARY
↓
AREA
↓
DIMENSIONS
↓
PLAN
↓
SECTION
↓
ELEVATION
↓
SCHEDULE
↓
DOCUMENT PACKAGE

Only affected derived artifacts should be regenerated where deterministic dependency analysis permits.

33. DIRTY STATE

Derived objects may receive a dirty state.

Initial states:

CURRENT
DIRTY
GENERATING
VALID
INVALID
SUPERSEDED

A model change may mark dependent documentation:

DIRTY

rather than immediately deleting the previous artifact.

34. DOCUMENTATION SERVICE

The documentation service converts the computational model into architectural documentation.

Responsibilities include:

view generation
drawing generation
sheet composition
dimensions
annotations
schedules
title blocks
revision blocks
sheet indexing
document assembly

The documentation engine consumes the model.

It does not become the source of truth.

35. VIEW GENERATION

A view is derived from:

model_version_id

view definition
visibility rules
cut plane
orientation
scale
discipline

The resulting view becomes an input to drawing generation.

36. DRAWING GENERATION PIPELINE

The drawing pipeline becomes:

```
MODEL VERSION
↓
VIEW GENERATION
↓
GEOMETRY EXTRACTION
↓
ANNOTATION GENERATION
↓
DIMENSION GENERATION
↓
SCHEDULE GENERATION
↓
SHEET COMPOSITION
↓
DRAWING VALIDATION
↓
EXPORT
```

37. ARCH D SUPPORT

The initial documentation stack shall support:

ARCH D

as an explicit sheet format.

The sheet template system shall control:

paper dimensions
orientation
title block
project information
drawing number
drawing title
revision information
graphic standards

38. COMPLIANCE ENGINE

The compliance subsystem shall remain independent from Aria.

The compliance engine receives:

project model
jurisdiction
code manifest
applicable rules

and produces:

requirements
findings
exceptions
validation status
provenance

39. REGULATORY PROVENANCE

Every regulatory finding must be traceable to:

code_manifest_id
code_source_id
code_rule_id
project_id
revision_id
model_version_id

A generated compliance result without provenance shall not be considered complete.

40. VALIDATION ENGINE

Validation shall operate across multiple domains.

Initial validation categories:

CODE
SPATIAL
GEOMETRIC
ACCESSIBILITY
FIRE
STRUCTURAL_COORDINATION
DOCUMENT
MODEL

The validation engine shall produce deterministic result records wherever the underlying rule is deterministic.

41. VALIDATION GATE

Final document generation shall be subject to validation status.

Conceptually:

```
MODEL
↓
VALIDATION
↓
PASS / CONDITIONAL / FAIL
↓
DOCUMENT GENERATION
```

A failed mandatory validation shall prevent a project from entering final acceptance state.

42. REDLINE PIPELINE

Redlines become application commands rather than isolated annotations.

The workflow is:

```
REDLINE CREATED
↓
INTERPRETED
↓
COMMAND PROPOSED
↓
COMMAND VALIDATED
↓
MODEL MODIFIED
↓
VALIDATION
↓
DRAWINGS REGENERATED
↓
REDLINE RESOLVED
```

The redline remains linked to the resulting revision.

43. EVENT BUS

Application events shall be emitted after successful transactional operations.

Initial events:

ProjectCreated

RequirementCreated
ModelChanged
RevisionCreated
AriaCommandProposed
AriaCommandExecuted
GeometryGenerated
ValidationCompleted
DrawingGenerated
RedlineCreated
RedlineResolved
DocumentPackageGenerated
AcceptanceRecorded

Events are integration signals and audit inputs.

44. EVENT RULE

Events shall never be emitted as successful completion signals before the associated transaction has committed.

The sequence is:

```
TRANSACTION
↓
COMMIT
↓
EVENT PUBLICATION
```

This prevents external consumers from observing a state that was subsequently rolled back.

45. OUTBOX PATTERN

To preserve reliable event publication, OneDraft shall use an outbox pattern.

Within the same database transaction:

```
MODEL CHANGE
+
OUTBOX EVENT
```

are committed together.

A background publisher then sends the event to the event system.

This prevents the failure condition:

```
DATABASE COMMIT
+
EVENT LOST
```

46. OBSERVABILITY

The application shall implement:

- structured logging
- metrics
- distributed tracing
- request IDs
- job IDs
- project IDs
- revision IDs
- model version IDs

Every major operation must be traceable across API and worker boundaries.

47. LOGGING RULES

Logs shall contain operational identifiers but must not expose sensitive project content unnecessarily.

Preferred fields:

- timestamp
- level
- service
- request_id
- job_id
- project_id
- revision_id
- operation
- duration
- status
- error_code

48. ERROR TAXONOMY

Initial application errors shall include:

- AUTHENTICATION_ERROR
- AUTHORIZATION_ERROR
- PROJECT_NOT_FOUND
- REVISION_CONFLICT
- MODEL_VERSION_CONFLICT
- INVALID_COMMAND
- CONSTRAINT_CONFLICT
- OBJECT_NOT_FOUND
- VALIDATION_FAILED
- JOB_NOT_FOUND

JOB_FAILED
DOCUMENT_GENERATION_FAILED
CODE_MANIFEST_INVALID
DEPENDENCY_CONFLICT

Errors shall be machine-readable.

49. RETRY POLICY

Retries shall be permitted only for operations classified as retry-safe.

Transient infrastructure failures may be retried.

Domain validation failures shall not be retried automatically.

Examples of retryable conditions:

temporary database connection failure
temporary object-storage failure
temporary queue failure
temporary AI provider failure

Examples of non-retryable conditions:

invalid geometry
constraint conflict
revision conflict
authorization failure
invalid command
failed mandatory validation

50. AI RETRY POLICY

AI requests shall receive deterministic execution identifiers where practical.

A retry must not automatically execute a previously proposed command twice.

The sequence remains:

AI RESPONSE
↓
COMMAND IDENTIFICATION
↓
COMMAND VALIDATION
↓
COMMAND EXECUTION

51. CONFIGURATION

Runtime configuration shall be environment-driven.

Initial configuration domains:

DATABASE_URL
API_HOST
API_PORT
QUEUE_URL
OBJECT_STORAGE
AUTH_CONFIGURATION
AI_PROVIDER_CONFIGURATION
LOG_LEVEL
ENVIRONMENT

Secrets must never be committed to source control.

52. ENVIRONMENT MODEL

Initial environments:

development
test
staging
production

Each environment shall maintain independent:

database
object storage
queues
credentials
AI configuration

Development data must never be confused with production project data.

53. LOCAL DEVELOPMENT

The complete OneDraft stack shall eventually run locally through container orchestration.

Conceptually:

docker compose
↓
postgres
redis / queue
api
worker
web

object storage

The exact queue and object-storage implementations remain infrastructure choices behind their interfaces.

54. CONTAINERIZATION

Each executable application shall have an independently deployable container.

Initial services:

onedraft-web
onedraft-api
onedraft-worker

Supporting infrastructure:

postgres
queue
object-storage

The application containers shall remain stateless where possible.

55. STATEFUL SERVICES

Persistent state belongs to:

PostgreSQL
Object Storage
Job/Queue persistence where required

API and worker containers should not depend upon local filesystem state for authoritative project data.

56. WORKER ISOLATION

Workers must identify the exact computational target before execution.

A worker job shall contain:

project_id
revision_id
model_version_id
job_id
input_hash

If the project state has advanced beyond the expected version, the worker must detect the mismatch before publishing results.

57. STALE RESULT PROTECTION

A worker result must never silently replace newer project state.

The worker must verify:

`expected_model_version`

against current state.

If the state differs:

`STALE_RESULT`

The result may be retained for diagnostics but must not become current state.

58. SECURITY BOUNDARY

The application security architecture shall distinguish:

`identity`
`authorization`
`project ownership`
`application capability`
`data access`

Possession of an AI capability does not grant unrestricted project access.

59. CAPABILITY MODEL

Aria and internal services shall receive explicit capabilities.

Examples:

`READ_PROJECT_MODEL`
`READ_REQUIREMENTS`
`PROPOSE_MODEL_CHANGE`
`EXECUTE_MODEL_COMMAND`
`RUN_VALIDATION`
`GENERATE_DRAWING`
`GENERATE_DOCUMENT`

Capabilities shall be evaluated by the application layer.

60. DIRECT DATABASE ACCESS

The following components shall not receive unrestricted direct database credentials:

frontend
external integrations
AI provider
client applications
browser code

Database access remains an internal infrastructure concern.

61. API CLIENT GENERATION

The OpenAPI contract established previously becomes the source for generated clients.

Initial generated client:

packages/contracts/typescript-client

The web application shall consume the generated API contract.

Python services may consume generated or shared typed models where appropriate.

62. PYDANTIC CONTRACTS

The Python domain/API layer shall provide typed request and response models.

Initial categories:

ProjectCreate
ProjectResponse
RequirementCreate
ElementCreate
CommandRequest
CommandResponse
RevisionResponse
RedlineCreate
ValidationRequest
ValidationResponse
DrawingGenerationRequest
DocumentGenerationRequest
JobResponse
AcceptanceRequest

These models must correspond to the OpenAPI contract.

63. CONTRACT CONSISTENCY

The following must remain synchronized:

- OpenAPI
- Pydantic models
- TypeScript client
- application commands
- domain commands

Contract drift becomes a build or integration failure rather than an accepted condition.

64. TESTING ARCHITECTURE

Testing shall operate at multiple levels.

UNIT
↓
DOMAIN
↓
APPLICATION
↓
INTEGRATION
↓
API
↓
WORKER
↓
END-TO-END

65. DOMAIN TESTS

Domain tests shall verify:

- element creation
- relationship creation
- constraint enforcement
- model mutation
- revision creation
- dependency propagation
- invalid operation rejection

These tests shall not require PostgreSQL.

66. APPLICATION TESTS

Application tests shall verify:

- authorization
- transaction boundaries
- repository interaction
- revision handling
- command execution
- job creation
- event creation

67. DATABASE INTEGRATION TESTS

Database tests shall verify:

- migrations
- foreign keys
- unique constraints
- transactions
- rollback behavior
- revision persistence
- event persistence
- idempotency

68. API TESTS

API tests shall verify:

- authentication
- authorization
- request validation
- response contracts
- HTTP status codes
- error schemas
- pagination
- idempotency

69. WORKER TESTS

Worker tests shall verify:

- job acquisition
- job execution
- version checking
- retry behavior

failure handling
result persistence
stale-result protection

70. ARIA TESTS

Aria integration tests shall distinguish:

AI interpretation

from:

command validity

A test shall never assume that an AI response is valid merely because it is syntactically correct.

The command validator remains authoritative.

71. END-TO-END TEST

The first executable end-to-end stack test shall extend the prior database/API integration test.

The sequence becomes:

1. Start PostgreSQL
2. Start queue
3. Start object storage
4. Start API
5. Start worker
6. Create organization
7. Create user
8. Authenticate
9. Create project
10. Create requirements
11. Create building
12. Create levels
13. Create spaces
14. Create walls

15. Create doors
 16. Create windows
 17. Generate model version
 18. Generate plan views
 19. Generate elevations
 20. Generate section
 21. Generate ARCH D sheets
 22. Submit Aria modification
 23. Validate command
 24. Execute command
 25. Create new revision
 26. Generate geometry
 27. Regenerate affected drawings
 28. Create redline
 29. Resolve redline
 30. Run compliance validation
 31. Run document validation
 32. Generate final document package
 33. Hash document package
 34. Record acceptance
 35. Retrieve complete project history
 36. Verify provenance chain
-

72. PROVENANCE TEST

The system shall verify:

PROJECT

↓

MODEL VERSION

↓

REVISION

↓

COMMAND
↓
CHANGE RECORD
↓
GEOMETRY
↓
CODE MANIFEST
↓
VALIDATION
↓
DRAWING
↓
DOCUMENT PACKAGE
↓
ACCEPTANCE

Every link must remain discoverable.

73. REBUILD TEST

The system must be capable of reconstructing a historical project state from:

project_id
revision_id
model_version_id

The reconstructed state must produce equivalent domain state.

This becomes a mandatory regression test.

74. DETERMINISTIC REGENERATION

Given identical:

model_version
view definition
sheet template
documentation configuration

the documentation engine should produce equivalent documentation content.

Where byte-level determinism is impractical, semantic equivalence and provenance hashes shall be validated.

75. APPLICATION PACKAGE STRUCTURE

The initial implementation repository should evolve toward:

```
OneDraft/
├── README.md
├── LICENSE
├── pyproject.toml
├── .env.example
├── docker-compose.yml
├── apps/
│   ├── web/
│   ├── api/
│   └── worker/
├── packages/
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   ├── geometry/
│   ├── compliance/
│   ├── documentation/
│   ├── aria/
│   ├── events/
│   └── contracts/
├── db/
│   └── migrations/
├── docs/
│   ├── architecture/
│   ├── openapi/
│   └── specifications/
└── tests/
    ├── unit/
    ├── integration/
    ├── api/
    ├── workers/
    └── e2e/
```

76. IMPLEMENTATION ORDER

The executable application stack shall be implemented in the following order:

1. Repository initialization
2. Configuration system
3. Database connection layer
4. Repository interfaces

5. PostgreSQL repositories
 6. Transaction manager
 7. Domain models
 8. Application services
 9. Project Model command service
 10. API dependency injection
 11. API routes
 12. Authentication boundary
 13. Job abstraction
 14. Worker runtime
 15. Event/outbox system
 16. Object storage adapter
 17. Aria orchestration boundary
 18. Geometry worker
 19. Compliance worker
 20. Validation worker
 21. Documentation worker
 22. Web client
 23. End-to-end integration
-

77. FIRST CODE PACKAGE

The first implementation package following the persistence/API artifacts shall therefore contain:

```
packages/domain/  
packages/application/  
packages/infrastructure/  
apps/api/  
apps/worker/
```

The purpose is to transform the static contract into an executable application boundary.

78. FIRST APPLICATION SERVICE

The first service shall be:

ProjectModelService

Its responsibility is to establish the canonical mutation pathway.

Initial operations:

```
create_project
create_building
create_level
create_space
create_element
create_relationship
create_constraint
execute_command
create_revision
```

79. FIRST REPOSITORY IMPLEMENTATION

The first repositories shall be:

```
ProjectRepository
ModelRepository
RevisionRepository
CommandRepository
EventRepository
```

Additional repositories are added as each application capability becomes executable.

80. FIRST API ROUTES

The first operational endpoints shall be:

```
POST /api/v1/organizations
POST /api/v1/projects
GET  /api/v1/projects/{project_id}
GET  /api/v1/projects/{project_id}/model
POST /api/v1/projects/{project_id}/commands
GET  /api/v1/projects/{project_id}/revisions
GET  /api/v1/projects/{project_id}/events
```

These routes establish the minimum executable project-model loop.

81. FIRST WORKER JOB

The first worker operation shall be:

MODEL_VALIDATION

This establishes the complete pattern:

API
↓
JOB
↓
WORKER
↓
MODEL
↓
RESULT
↓
EVENT

The same pattern can subsequently support geometry, compliance, drawings, and documents.

82. FIRST ARIA INTEGRATION

Aria shall initially operate in:

PROPOSE

mode.

The initial sequence shall be:

USER INSTRUCTION
↓
ARIA
↓
STRUCTURED COMMAND
↓
COMMAND VALIDATOR
↓
PROPOSED COMMAND

Execution remains an application-controlled operation.

Once the command boundary has been proven, execution can be automated according to project authorization and validation rules.

83. FIRST GEOMETRY INTEGRATION

The first geometry operation shall be deliberately simple and deterministic.

Example:

MOVE_ELEMENT

The system shall prove that:

```
command
↓
domain mutation
↓
geometry update
↓
validation
↓
revision
```

can be executed without stale geometry becoming current state.

84. FIRST DOCUMENTATION INTEGRATION

The first documentation operation shall generate a basic:

ARCH D

sheet from the computational project model.

The first sheet shall establish:

```
project title
sheet number
drawing title
scale
model-derived geometry
dimensions
annotations
revision
```

This proves the fundamental OneDraft principle:

```
MODEL
↓
DRAWING
```

rather than:

```
AI
↓
```

85. STACK COMPLETION CRITERIA

The application runtime layer shall be considered operational when the system can:

```
accept an authenticated request
↓
load a project
↓
load its current model
↓
execute a domain command
↓
persist the change
↓
create a revision
↓
emit an event
↓
queue a computational job
↓
execute the job
↓
validate the result
↓
persist derived artifacts
↓
return the result through the API
```

86. ARCHITECTURAL INVARIANTS

The following principles are now permanent OneDraft architecture rules.

The database is not the domain model.

The API is not the domain model.

Aria is not the domain model.

A drawing is not the project model.

A PDF is not the project model.

AI output is not authoritative until validated through application capabilities.

Derived artifacts must identify the model state from which they were generated.

Historical project states remain reconstructable.

Regulatory findings remain provenance-linked.

No asynchronous worker may silently overwrite newer project state.

No external client may bypass the application boundary to modify project state.

87. CURRENT ONEDRAFT STACK

The architecture now consists of:

USER EXPERIENCE
↓
WEB APPLICATION
↓
OPENAPI / REST API
↓
APPLICATION SERVICES
↓
DOMAIN MODEL
↓
PROJECT MODEL SERVICE
↓
POSTGRESQL

with computational execution:

APPLICATION SERVICES
↓
JOB SYSTEM
↓

ARIA	GEOMETRY	COMPLIANCE	DOCUMENTATION
------	----------	------------	---------------

↓
VALIDATION
↓
DERIVED STATE

and persistent provenance:

PROJECT
↓
MODEL
↓
REVISION
↓
EVENT
↓
DERIVED ARTIFACT
↓
ACCEPTANCE

88. VERSION 0.1 ENGINEERING STATE

OneDraft now has the following established engineering layers:

Product Definition

System Architecture

Technology Architecture

Domain Model

Persistence Model

OpenAPI Contract

Versioning Model

AI Command Boundary

Regulatory Model

Drawing Model

Redline Model

Validation Model

Document Provenance

Acceptance Workflow

Application Service Architecture

Repository Architecture

Transaction Boundary

Worker Architecture

Job Model

Event / Outbox Architecture

Object Storage Boundary

Observability Architecture

Testing Architecture

Container Runtime Architecture

89. NEXT IMPLEMENTATION ARTIFACTS

The next executable package shall consist of:

Artifact A

packages/domain/

The canonical Python domain model.

Artifact B

packages/application/

The application command and query services.

Artifact C

packages/infrastructure/

PostgreSQL repositories, transaction management, object storage, jobs, and infrastructure adapters.

Artifact D

apps/api/

The first executable FastAPI application implementing `/api/v1`.

Artifact E

apps/worker/

The asynchronous worker runtime.

Artifact F

tests/integration/

The first database/application integration suite.

90. FINAL ARCHITECTURAL DECISION

The OneDraft stack shall now be treated as an executable computational system.

The fundamental execution loop is:

```
USER
↓
ARIA / API
↓
COMMAND
```

↓
DOMAIN
↓
TRANSACTION
↓
PROJECT MODEL
↓
REVISION
↓
EVENT
↓
JOB
↓
COMPUTATION
↓
VALIDATION
↓
DERIVED DOCUMENTATION
↓
CUSTOMER REVIEW
↓
ACCEPTANCE

The project model remains authoritative throughout the lifecycle.

Aria interprets intent.

Application services control execution.

The domain enforces semantic rules.

PostgreSQL preserves transactional state.

Workers perform expensive computation.

The regulatory engine verifies applicable requirements.

The documentation engine produces architectural documentation.

The event system preserves operational history.

The acceptance layer records the customer-controlled completion state.

91. SPECIFICATION STATUS

ONEDRAFT APPLICATION RUNTIME, SERVICE ARCHITECTURE & EXECUTION STACK v0.1

STATUS: ESTABLISHED

The persistence/API layer is established.

The application/runtime architecture is now established.

The next implementation stage is the conversion of this architecture into the executable Python package structure, beginning with:

```
packages/domain/  
packages/application/  
packages/infrastructure/  
apps/api/  
apps/worker/  
tests/integration/
```

The immediate implementation objective is therefore:

```
DOMAIN  
↓  
APPLICATION  
↓  
INFRASTRUCTURE  
↓  
API  
↓  
WORKER  
↓  
INTEGRATION TEST
```

This constitutes the next executable layer of the OneDraft software stack.